

Assignment 4: GUI Testing

Team Members:

Robin Sanssi (rosa24)

Mirnes Mrso (mimr24)

Group Number:35

Repository Link:

<https://github.com/mirNNes/bsv-edutask>

1. Graphical User Interface Tests

Test design

We applied a simplified version of the test design technique for the GUI tests by identifying actions, conditions, combinations, and expected outcomes.

Step 1: Identify actions and conditions

The actions tested are:

- Create to-do item : successful / unsuccessful
- Toggle to-do item : successful/ unsuccessful
- Delete to-do item : successful / unsuccessful

The conditions that affect the outcome are:

Condition	Possible values
To-do text	valid / empty
To-do item exists	yes / no
User interaction	create / toggle / delete

Step 2: Construct combinations

ID	Action	Conditions	Expected outcome
1	Create to-do item	valid text	item appears in the list
2	Toggle to-do item	item exists	item changes completed state
3	Delete to-do item	item exists	item is removed from the list

Step 3: Denote expected outcome

- For creating a to-do item, the action is successful if the user enters valid text and submits it.
- For toggling a to-do item, the action is successful if a to-do item exists and the user interacts with it.
- For deleting a to-do item, the action is successful if a to-do item exists and the user clicks the delete button.

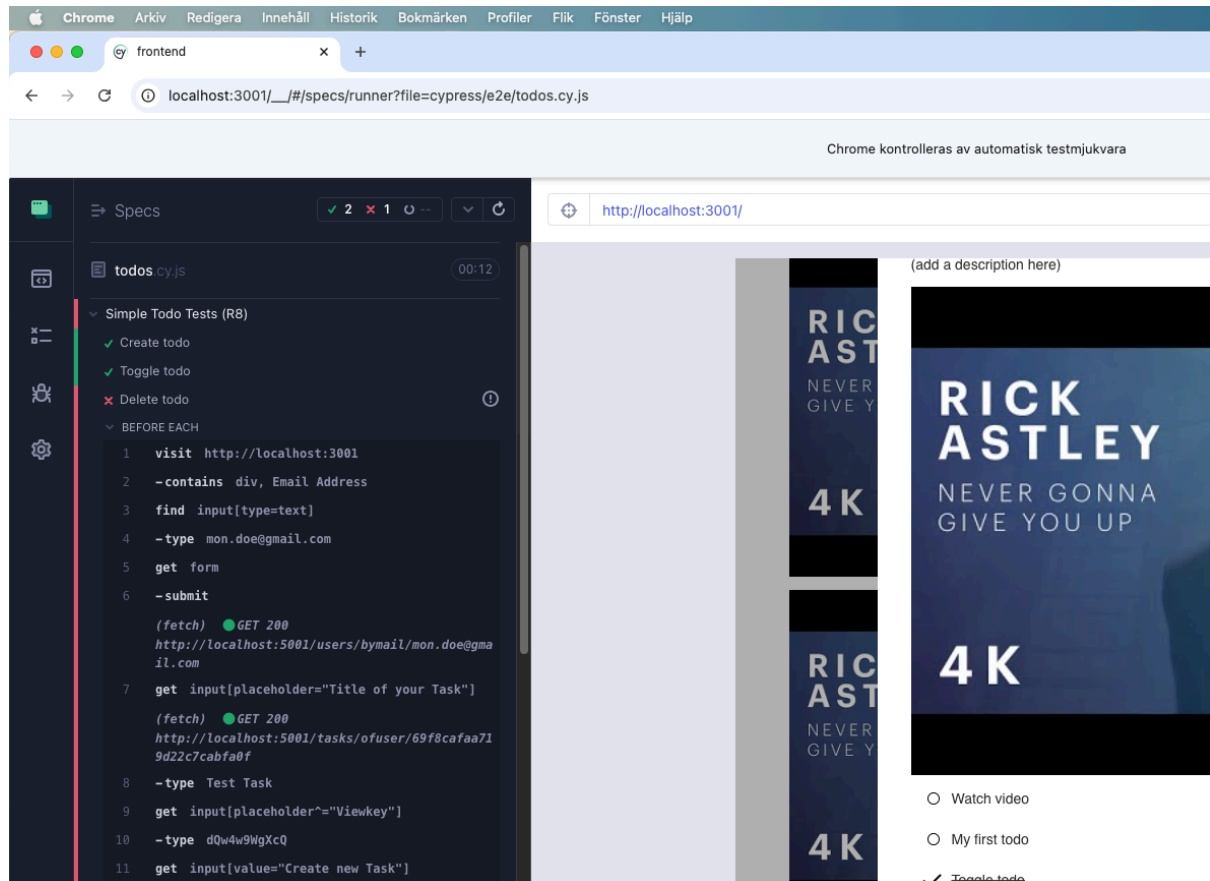
Step 4: Collapse to relevant test cases

Since each GUI requirement focuses on a single user action, the relevant test cases are:

Requirement	Test case	Expected result
R8UC1	Create to-do item with valid text	The item should appear in the list
R8UC2	Toggle an existing to-do item	The item should change completed state
R8UC3	Delete an existing to-do item	The item should be removed from the list

Test execution results:

- R8UC1 (Create to-do item): Passed
- R8UC2 (Toggle to-do item): Passed
- R8UC3 (Delete to-do item): Failed



Failure description

The delete functionality does not work correctly. After clicking the remove button, the todo item remains visible in the GUI. The Cypress log does not show any DELETE request being sent to the backend, indicating that the GUI does not properly trigger the deletion action.

2. Declarative vs. imperative UI Testing

1. Explanation

Declarative UI testing means describing what the expected behavior or outcome of the system is, without specifying every step in detail. The focus is on the result. For example, a test might express that a todo item should exist after being created, without explicitly describing every UI interaction step.

Imperative UI testing means describing exactly how the test should be executed step by step. It includes all actions the user performs, such as clicking buttons, typing into fields, and navigating through the interface.

2. Discussion

For UI testing, a combination of both approaches is often most useful. However, declarative testing is generally more suitable at a higher level because it makes tests easier to read, understand, and maintain.

Declarative tests focus on expected behavior and align well with user requirements. This makes them more stable when the UI changes. Imperative tests are necessary to interact with the GUI, but they can become fragile if the interface changes.

In practice, a good approach is to write test cases in a declarative way while implementing the interactions imperatively. This balances readability and control, especially when using tools like Cypress.